



Course of

---

SUPERVISOR

---

CO-SUPERVISOR

---

CANDIDATE

Academic Year

## **Abstract**

This thesis develops and evaluates a genre-based music recommendation system designed to enhance user experience on streaming platforms through feature similarity and users' historical preferences. With the increasing reliance on media platforms and the significant role of user preference in audio streaming services, this research integrates recommendation algorithms to predict user behavior and tailor content effectively. A novel approach of this system is its incorporation of a genre-filtering feature that encourages users to explore music from unfamiliar genres, thus broadening their musical horizons. The study primarily utilizes a dataset from Spotify, from Kaggle, employing algorithms that consider a variety of musical features such as danceability, tempo, and acousticness to formulate recommendations. The system utilizes TF-IDF and cosine similarity techniques in order to calculate a similarity index, the primary parameter to measure feature similarity and generate music recommendations. Evaluation of the system through user surveys indicates a positive reception, with high ratings for recommended songs suggesting successful customization. Despite limitations such as data bias and the simplicity of genre classification, the results affirm the system's potential in improving user engagement and satisfaction. This research contributes to advancing music recommendation systems by integrating more nuanced user preference modeling and offering opportunities for further enhancement in personalized music discovery.

## Table of Contents

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>I. Introduction</b>                                  | <b>2</b>  |
| <b>II. Literature Review</b>                            | <b>3</b>  |
| <b>III. Methodology and Development</b>                 | <b>9</b>  |
| <b>V. Discussion</b>                                    | <b>18</b> |
| i. Evaluating the Recommendation System                 | 18        |
| ii. Limitations                                         | 18        |
| iii. Area of Further Research                           | 19        |
| <b>VI. Conclusion</b>                                   | <b>20</b> |
| <b>VII. Reference list</b>                              | <b>21</b> |
| <b>VIII. Appendices</b>                                 | <b>23</b> |
| Appendix 1 - Files (Appendix 1.1, 1.2, 1.3, 1.4 & 1.5 ) | 23        |
| Appendix 2 - Website                                    | 23        |

## **I. Introduction**

It is undoubtedly said that the dependence of humanity on technological applications and specifically, media platforms has been on an ever-continuing rise today. This rise can be credited to several advancements that have integrated personalization as the primary strategy for increasing the usability of certain media applications, using the user's digital footprint (record data of user activity) to further predict their behavior and ultimately, tailoring contents to their liking. Algorithmically, these predictions of preference, or recommendations, have been implemented as recommendation systems (Aggarwal and Springer International Publishing Ag, 2018).

One of the industries where preference (or 'taste') seems to play a considerable role is the audio streaming services, such as Spotify. I consider myself particularly acquainted with Spotify due to its popularity and user-friendly interface, making it one of the leading platforms in the audio streaming industry (Duarte, 2024). In fact, the usability of audio streaming platforms are directly correlated to the engagement of users with the application by continuing to use the platform to listen to music of previous liking, and more importantly, discover new music that they may like. This is particularly important because unlike the streaming of previously existing music, exploring new music opens up the users to a vast variety of genres and artists, further diversifying their music preferences and keeping the users engaged with the platform in the long term. This can be useful in re-defining the relevant market by minimizing competitive substitutes to music streaming platforms as when introduced with variety, the users are less likely to get bored and switch to another form of entertainment. Bluntly, it is more profitable to Spotify if the switch of entertainment of the users is from streaming the music of one genre to another over a switch from listening to music to watching a movie on Netflix, thereby increasing the platform 'stickiness' (Grigorescu, 2020).

Although Spotify has a well-established recommendation system, there exists minimal algorithms where recommendation systems enhance the result for a particular search made by the user. For example, when a user searches 'Pop' genre on Spotify, or any media platform for that matter, the primary factor influencing the results is the popularity index, i.e. the most popular Pop song of the week would likely be recommended. My personal interest and knowledge with music has often led me to derive several recommendations for those around me who want to explore a new genre. I noticed that knowing about a person's prior music taste helped me greatly in understanding what kind of music in a particular genre they would be into. This entire process can be done rather efficiently through a similarity checking algorithm for a recommendation system. Therefore, there is a missed opportunity where the previously mentioned personalization can be added to enhance the quality of the results, or what if the user's search for Pop recommended the song that the user is most likely to enjoy, instead of basing it off of mere popularity, or other independent factors. Therefore, the research objective of this thesis is to construct a genre-based music recommendation system which utilizes the users' music taste (from their personal playlists) to create recommendations based on feature similarity with the users' pre-existing preferences.

## II. Literature Review

In order to understand Music recommendation systems, we must first understand the nature of music first which can be defined through several features such as the *danceability*, *tempo*, *loudness*, *instrumentalness*, etc. These features, defined numerically, allow us to take an algorithmic approach towards making a judgment of preference through measuring and comparing the features of multiple songs and interpret these measures as a similarity index. Therefore, multiple music streaming services use these features to build recommendation systems that suggest songs to the listeners. Music recommendation systems are a specific application of recommender systems designed to suggest musical content (songs, artists, albums, playlists) to users. These systems forecast and recommend songs or artists that the user is likely to appreciate based on their analysis of usage habits, preferences, and interactions. The goal here is to create a customised listening experience that maintains user satisfaction and engagement (Schedl et al., 2015).

A recommendation system can also be perceived to function as a filtering mechanism, estimating the weight a user would give to a specific item—in this case, a song. It is a fundamental component of large engines that operate using certain recommender algorithms to provide consumers with a single or a collection of suggestions based on such predictions (ElifTech Insights, 2021). All recommendation systems are primarily categorized into content-based filtering and collaborative filtering, the understanding of which is crucial for this project. These systems use different data sources and algorithms to predict and match user preferences with relevant items.

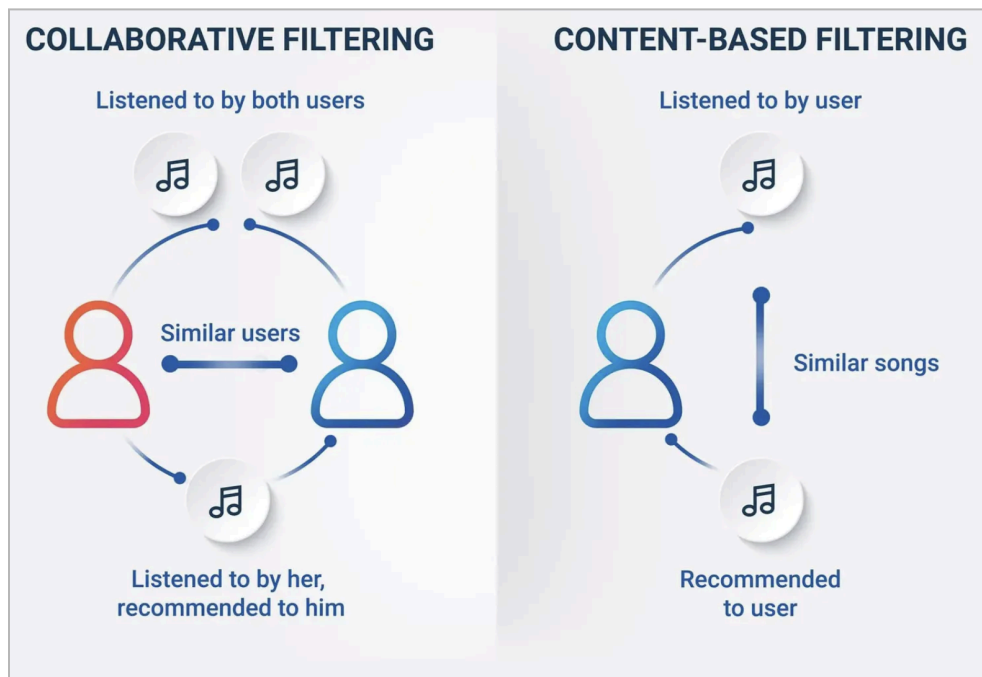


Figure 1: Collaborative Filtering vs Content-based Filtering (ElifTech Insights, 2021)

Content-based filtering operates by analyzing the attributes of items within a dataset and aligning them with a user's preferences, employing a model that predicts user engagement based on item feature similarity. This approach makes the assumption that if a user exhibits interest in a particular item, they are likely to appreciate other items with identical characteristics. In practice, this method emphasizes a

detailed feature extraction and analysis process, where both items and user preferences are represented as vectors in a multidimensional feature space (Ricci, Rokach and Shapira, 2015).

The first step of content-based filtering is item representation, where explicit features (genre, artists, etc.) and implicit features (tempo, danceability, etc.) play a key role in defining the nature of each item (songs, movies, etc.). While explicit features are relatively straightforward to gather, implicit features can be extracted in a more sophisticated manner using diverse techniques such as term frequency-inverse document frequency (TF-ID) for textual content (Baena-García et al., 2011) and mel-frequency cepstral coefficients (MFCCs) for audio analysis (Giannakopoulos and Pikrakis, 2014). These implicit features are readily available on open-source repositories such as Kaggle, owing to the community contribution. Next, feature selection allows us to identify the features that bear the most predictive power for user preferences since all features extracted cannot be perceived to be of equal importance. After identifying the relevant features, the next step is to encode these features as vectors. To define Vectorisation, it is a key concept which involves the transition of the formerly mentioned vectors into a numerical form assigned to each feature, reflecting the characteristics of the item (Ricci, Rokach and Shapira, 2015).

After dealing with item representation, the focus was shifted to user profile building and analysis. User profile is determined by the interactions that the users have had with the items (reviews, listening history, etc) by aggregating the feature vectors of items the user has liked or spent time engaging with. A user profile is similarly represented as a vector in the same feature space. Once the items and users have been well-represented, the system determines the similarity between the item vectors and the user's profile vector in order to suggest new items to the user by taking the item vectors and comparing them using similarity measures. Some of these methods are cosine similarity, where the cosine of the angle between two vectors reflects their similarity, or Euclidean distance, where the geometric distance between points in the feature space is calculated (Jure Leskovec, Anand Rajaraman and Ullman, 2020). These computations vividly illustrate how closely the user's preferences are aligned with the item's features. Items that have a higher degree of similarity to the user's profile vector are considered to be of interest to the user more likely and are thereby recommended. The user can receive the most relevant suggestions from the system by having these items ranked according to similarity ratings (Aggarwal and Springer International Publishing Ag, 2018).

Collaborative filtering on the contrary analyzes users' community interaction data to predict content preferences. This approach towards recommendation systems groups users and makes the assumption that if an item is enjoyed by a user, it will most likely also be enjoyed by users in the same group as him. The methodology operates on the principle that individuals with similar tastes in the past will continue to do so in the future (Ricci, Rokach and Shapira, 2015). This approach adds a human aspect to the recommendation systems so that the results are not purely based on algorithmic predictions, but rather from other users that share similar preferences.

| Persons \ Movies | Joe | Bob | Carol |
|------------------|-----|-----|-------|
| Titanic          | 5   | 1   | 5     |
| The reader       | 1   | 5   | 2     |
| Harry potter     | 4   | 2   | ?     |

Table 1: Recommendation process example (Thorat, Goudar and Barve, 2015)

Table 1 is an example where the people are users, and the movies are the items. To understand Carol's potential positive opinion about Harry Potter, one could examine her similarities with Joe. Another approach is to observe the similar patterns in ratings for Titanic and Harry Potter, indicating that individuals who enjoyed the former may also appreciate the latter. Table 1 provides an illustrative example of collaborative filtering (Thorat, Goudar and Barve, 2015).

In order to analyze the similarity amongst users, collaborating filtering uses the users-item interactions matrix to gather data regarding similarities amongst users or items. Therefore, collaborative filtering can be further classified into two aspects: user-based collaborative filtering, and item-based collaborative filtering (Aggarwal and Springer International Publishing Ag, 2018).

User-based filtering makes recommendations for items that identical users have previously enjoyed by identifying the preferences of similar users based on their interactions. The logic is if a user has a similar rating pattern to another user who likes an item that the former has not seen yet, the system may recommend that item by predicting a similar pattern of likeability. In order to compute similarity between users, a user-item interaction matrix must be built, with users represented in the rows and items in the columns. The composition of the matrix are ratings (or any other measure of preference) (Risdiandita, 2021),

|         |                       |     |     |     |     |      |      |      |
|---------|-----------------------|-----|-----|-----|-----|------|------|------|
|         | product rated by user |     |     |     |     |      |      |      |
| user id | 1.0                   | 0   | 5.0 | 0   | 0   | 0    | 0    | 0    |
|         | 0                     | 3.0 | 0   | 0   | 0   | 0    | 11.0 | 0    |
|         | 0                     | 0   | 0   | 0   | 9.0 | 0    | 0    | 0    |
|         | 0                     | 0   | 6.0 | 0   | 0   | 0    | 0    | 0    |
|         | 0                     | 0   | 0   | 7.0 | 0   | 0    | 0    | 0    |
|         | 2.0                   | 0   | 0   | 0   | 0   | 10.0 | 0    | 0    |
|         | 0                     | 0   | 0   | 8.0 | 0   | 0    | 0    | 0    |
|         | 0                     | 4.0 | 0   | 0   | 0   | 0    | 0    | 12.0 |

Figure 2: User-item interaction matrix (Risdiandita, 2021)

The system finds a neighborhood of users who are similar to the target user after calculating the similarities using methods like cosine similarity, modified cosine similarity, or Pearson correlation coefficient. Users with the most comparable rating tendencies are concentrated in this neighborhood. This approach aggregates the ratings of the item from the user's neighborhood in order to determine the rating that the target user could assign to an unobserved item. This can be a simple average or a weighted average where identical users have a greater influence on the prediction. Further, a threshold can be set to determine the minimum level of similarity for a user to be classified as a part of the neighborhood.

Recommendations are then made for items with the highest predicted ratings (Aggarwal and Springer International Publishing Ag, 2018).

Meanwhile, item-based filtering computes similarities between items rather than users, often based on the co-occurrence of ratings (or other measures of preference) across users. The logic is that if a user likes an item, the system recommends other items that similar users have rated highly. The assumption here is that items liked by similar users are likely to be of interest. The system constructs a matrix capturing the similarity between pairs of items. This matrix might be based on the premise that items are similar if they are rated similarly by the same users (Aggarwal and Springer International Publishing Ag, 2018) or by features of these items (Risdiandita, 2021).

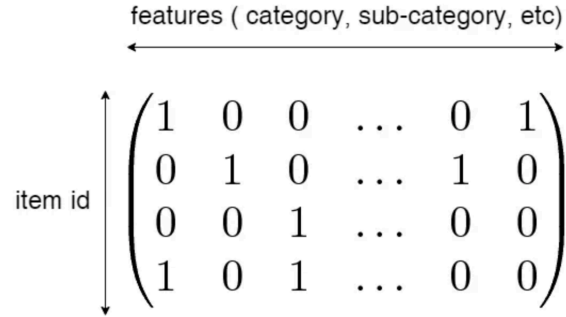


Figure 3: Item-feature matrix (Risdiandita, 2021)

Similar to user-based filtering, similarity metrics can be used for item and further, a weight based on a deterministic scale where similar items are weighed similarly. This is integral to predicting the rating a user might give to the item. The intuition is that items similar to those the user has liked will also be rated highly (Aggarwal and Springer International Publishing Ag, 2018).

Although collaborative filtering is effective at analyzing user interactions to provide recommendations, it very often than not struggles with the cold start problem, which makes it quiet challenging to promote new users or items with limited interaction histories. The cold start problem is a major obstacle faced by recommendation systems, including those used for music, movies, e-commerce, and various other domains where customized content is essential. This issue arises when the system cannot accurately recommend new things or users as it lacks sufficient data about them (Risdiandita, 2021). As the user-item matrix expands, this approach also has scalability problems that could result in computing inefficiencies. In contrast, content-based filtering faces issues of limited capacity to suggest items outside of a user's predetermined preference which results in a more restricted and less varied set of suggestions due to its heavy reliance on the availability and quality of item metadata. Moreover, content-based systems may find it difficult to fully capture the nuanced nature of user preferences because they depend overly on explicit item qualities, which might not adequately convey the motivations behind a user's musical preferences. Over-specialisation, a problem that occurs when recommendation systems continuously offer recommendations that closely align with a user's previous interactions or preferences, without exposing them to a wider range of possibilities is another limitation of content-based filtering (Dietmar Jannach, Cambridge University Press and Al, 2012).

The constraints of both the approaches are overcome by hybrid systems, combining the strengths of both collaborative filtering and content-based filtering methodologies to address their respective limitations. These systems hold the aim to merge the user behavior analysis essential to collaborative filtering with



the in-depth item attribute analysis of content-based filtering to provide a more sophisticated and comprehensive recommendation mechanism which stands out as an ideal choice of recommender systems, combining the advantages of both fundamental approaches. The operating essence of hybrid systems lies in their ability to mitigate the cold start problem of collaborative filtering associated with new users and items by incorporating content-based features that do not rely solely on user-item interactions. Conversely, they build upon the content-based strategy by incorporating diversity and spontaneity through collaborative signals, which helps to overcome the problem of over-specialization, causing recommendations to become unduly limited (Dietmar Jannach, Cambridge University Press and AI, 2012).

Hybrid recommendation systems can be implemented through several forms. Some of these approaches include combining the elements of both content-based and collaborative filtering through using collaborative filtering scores with content-based features. A few approaches that can be adapted for building hybrid systems are: Feature Combination, which combines the collaborative filtering's similarity metrics with the characteristics utilized in content-based filtering, making more accurate predictions through user-item interactions and content features, and Model Combination, which executes the collaborative and content-based filtering models first, independently and then merges their predictions using a variety of techniques. These techniques include voting systems and weighted averages, which assign a higher degree of trust to certain models based on context. Additionally, Unified Modeling is another approach which integrates content elements and user-item interactions in a more advanced manner than the formerly mentioned techniques. Methods such as matrix factorization can be modified to incorporate content features, allowing the model to learn undetected factors that represent both the user's preferences and the item's characteristics. Both forms of data can be processed and acquired simultaneously using machine learning frameworks like neural networks (Dietmar Jannach, Cambridge University Press and AI, 2012).

Many music streaming service providers use hybrid recommendation systems due to their increased efficiency, including Spotify, which has used several such sophisticated algorithms to enhance user experience. Using the hybrid recommendation system in particular allows Spotify to analyze the audio features of music tracks and use them as a tool to understand user behavior and preferences intricately. Spotify offers personalized music recommendations with a combination of natural language processing (NLP), audio analysis, and collaborative filtering. To find trends and preferences, it examines a tonne of data, such as music attributes, playlist building tendencies, and user behavior. Spotify generates customized playlists and music recommendations that align with individual preferences and cultural contexts by analyzing the connections between songs, user interactions, and contextual data, thereby making Spotify a prime example of efficiently implementing the hybrid system (Torabi, 2023). Our platform of interest for this particular study is Spotify as it conveniently allows us to integrate its Web API into python through the Spotipy Python library, facilitating full access to all music data provided by the Spotify platform about explicit features, such as artists and playlists, and also, implicit features, such as danceability, tempo, loudness, etc.

A 2022 research from the Royal Philharmonic Orchestra (RPO) showed a notable increase in the percentage of people who wished to expand their horizons and learn about new musical genres this summer - up from 62% to 78% since 2018. The RPO questioned 2,000 adults about their desire for a musical journey, specifically asking if they had any new music discoveries or genres they would like to learn more about this year. Although the shift towards music discovery was largely driven by young adults (ages 18 to 35) (87%), people of all ages showed interest in new musical genres, with 80% of those

over 35 and 70% of those over 55. Orchestral music ranked second behind pop music (31% and 33%, respectively) among the most popular music genres. Rock music was next at (27%) with R&B (23%) and dance music (20%). While R&B and dance were the most popular among young people (34% and 29%), rock was the most popular among those over 55 (33%). RPO research conducted over a two-year period during the COVID-19 pandemic revealed that music was present in nearly all houses throughout the home isolation phase (94%), with 25% of adults reporting a significant increase in radio listening. Across the country, 11% of respondents claimed to have kept up their connection to live music through internet broadcasts and recordings of performers giving performances from their homes, while 12% said they had read up on a specific musical interest or musician during a lockdown. The RPO saw a 290% rise in website traffic and a 755% increase in social media engagement during the lockdown period as more people turned to the internet to access live performances, recorded music, and educational resources (Wright, B, 2022). These figures demonstrate a clear trend: people's interest in exploring new genres is expanding. Incorporating algorithmic philosophies of recommendation systems into genre exploration optimizes the user experience by recommending songs that, although from a different genres, compliment the users' pre-established music taste to ensure the exploration doesn't completely disassociate listeners from their preferred music and thereby, mitigating the risk of user attrition. This gap in literature becomes the primary goal with this research.

### III. Methodology and Development

This thesis aimed to investigate the functionality and effectiveness of a recommendation system that incorporates a genre-filtering feature, specifically aimed at facilitating users' exploration of new music genres. The research focused on designing a system that can offer recommendations to ensure a transition for users looking to diversify their musical tastes into unfamiliar genres.

The first step towards this research was some necessary data collection. This study primarily deals with the Spotify dataset from Kaggle, comprising song data collected via the Spotify API (Malvani, 2022). The description of the dataset, comprising five .csv files, is as follows:

#### ➤ data.csv

##### ○ Variables:

- Valence, Acousticness, Danceability, Energy, Instrumentalness, Liveness, Loudness, Speechiness, Tempo: Musical features of songs that quantify mood, acoustic quality, rhythm, etc.
- Year, Release Date: The release year and exact date of the song.
- Artists, Name: The artist(s) and the name of the track.
- Duration\_ms: The length of the song in milliseconds.
- Explicit: Indicates whether the song contains explicit content.
- ID: Unique identifier for the song on Spotify.
- Key, Mode: Musical key and mode in which the song was composed.
- Popularity: A metric that quantifies the popularity of the song on Spotify.

#### ➤ data\_by\_artist.csv

##### ○ Variables:

- Artists: Name of the artist.
- Count: Number of tracks by the artist in the dataset.

- The dataset also includes averages of musical features per artist such as Acousticness, Danceability, Energy, etc., similar to those described in Song Data.
- data\_by\_genres.csv
  - Variables:
    - Genres: The genre classification.
    - Musical feature averages (Acousticness, Danceability, Energy, etc.) calculated across all tracks within a genre.
- data\_by\_year.csv
  - Variables:
    - Year: The year of the song's release.
    - Annual averages of musical features (Acousticness, Danceability, Energy, etc.) similar to those in Song Data.
- data\_w\_genres.csv
  - Variables:
    - Genres: List of genres associated with the artist.
    - Artists: Name of the artist.
    - Aggregated musical features of the artist's discography (Acousticness, Danceability, Energy, etc.).

In order to understand the dataset well, Python's Pandas, Seaborn and Matplotlib libraries were used to perform some necessary manipulation and analysis such as removing duplicates and providing statistical insights using graphs and plots (Figure 4 & 7).

```
song_data_df.drop_duplicates('artists_song', inplace = True)
song_data_df[song_data_df['name'] == 'Adore You']
```

*Figure 4: Removing Duplicates (Appendix 1.2)*

Further, the data was cleaned through parsing genres and artists from string representations to lists, deduplicating songs, and handling missing values. This was helpful later on in the project as the nature of the data was understood and removed any abnormalities that serve as an obstacle for the building of the recommendation system. Lastly, MinMaxScaler was applied to normalize numerical features and One-Hot Encoding (OHE) was implemented, for categorical variables such as year and popularity, to transform them into a format suitable for machine learning models (Appendix 1.2).

The next step was to construct the recommendation system and find a way to mathematically interpret playlists (of the users) which can then be used to numerically represent their similarity with other playlists. Therefore, the TF-IDF was used to transform genre lists into a TF-IDF matrix to quantify genre importance numerically. Floating-point columns were scaled to a 0-1 range, and also adjusted for a balanced feature representation. These processed features—genre importance, scaled numeric values, and

encoded categorical data—were then combined into a single DataFrame for the sake of preserving unique identifiers from the original dataset (Appendix 1.2).

Next, the recommendation system was constructed (Appendix 1.2). First, representing each song in the playlist as a vector, making its contents the features of the song (bass, instrumentality, etc.). These vectors were used to calculate the cosine similarity index (Figure 5) between each song based on their features to identify tracks with the same characteristics. Cosine similarity is a metric, helpful in determining, how similar the data objects are irrespective of their size.

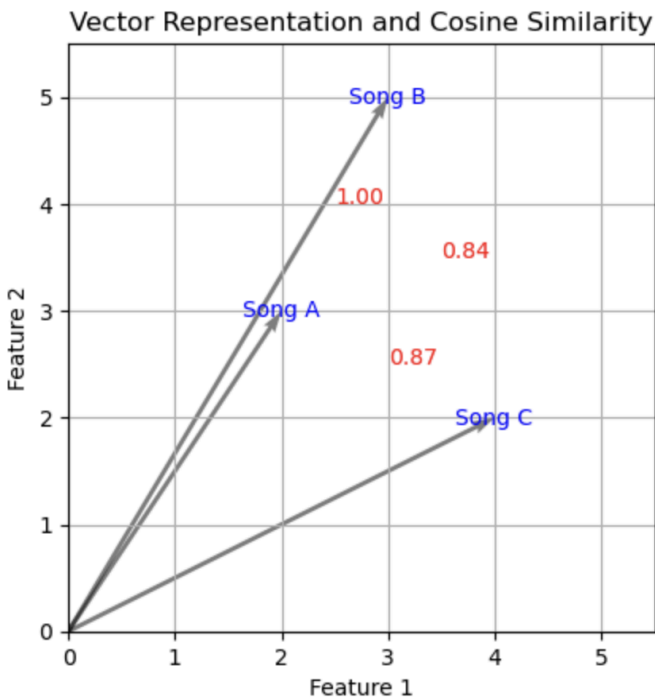


Figure 5: Cosine Similarity visualized.

Then playlist analysis was required. A user's playlist was turned into a weighted feature vector, This vector consisted of each song with their features, and recently added songs were given more importance to ensure that the recommendations were more recent.

Now that the algorithm had been constructed, it was necessary to find a way to add the genre filter initially discussed, ensuring that the recommendations made to users were limited to a particular genre of their interest, successfully aligning with the aim of genre-based filtering.

However, a problem was encountered: the genres provided by the dataset and the API were *sub-genres*, which were too narrow and specific. Using these would have significantly decreased the usability when the algorithm was implemented in practice, as the vast majority of listeners are not very familiar with sub-genres, and are more likely to recognize main genres (it is more likely that someone is familiar with 'Pop' rather than 'Afghan Pop'). Therefore, some feature engineering was performed to create the 'Main\_Genre' variable that derives the umbrella genre from the sub-genre. The determination of the main

genre from the sub-genre involved creating a list of most of the genres observed manually in the dataset and categorizing them into a list. A validation function was then created to classify the sub-genres based on the main genre they contain in their name ('Afghan Pop' was classified as 'Pop' since it included 'Pop' in its name, a rather obvious classification). The sub-genres that did not fall under any category were merely categorized into 'Other' (Figure 6).

```
genre_list = [
    "Pop", "Rock", "Indie", "Metal", "Classical", "Hip Hop", "Rap", "Folk", "Jazz", "Punk", "House", "Blues",
    "Soul", "Country", "Funk", "Reggae", "Techno", "Latin", "R&B", "Trance", "Ambient", "EDM",
    "Ska", "Gospel", "Disco", "Opera", "Grime", "Bass", "Drum"
]

def get_main_genres(genre_sublist):
    main_genres = []
    for subgenre in genre_sublist:
        for main_genre in genre_list:
            if main_genre.lower() in subgenre.lower():
                # Add the main genre to the list if not already included
                if main_genre not in main_genres:
                    main_genres.append(main_genre)
    return main_genres if main_genres else ["Other"]

song_data_df['Main_Genre'] = song_data_df['genre_lists'].apply(get_main_genres)

song_data_df.tail(3)
```

*Figure 6: Feature Engineering - Main\_Genre (Appendix 1.2)*

This completes the construction of the recommendation system.

In order to evaluate this recommendation system, a survey was conducted using Convenience Sampling. The respondents were required to fill in a survey form, entering the link of any playlist of the songs they liked, or their 'Spotify Wrapped' of the year, which is described as the "the receipt" of the songs that "you secretly couldn't stop streaming" (Spotify, 2023), an ideal representation of a person's taste in music. They were also required to enter the genre that they wanted to explore and their email, which they would receive their recommendation on. The survey was conducted through Google Form which was connected to an initial HTML page where the respondents consented to sharing their data and were instructed on how to fill in the form (Appendix 2). The HTML page, integrated with Google Forms, was uploaded to GitHub Pages, a site-hosting service provided by GitHub. The survey was sent to around 50 people and 38 responded. Their responses were put through the Recommendation System one-by-one and sent to their respective email account for ratings for each of the songs. These ratings were then recorded in an Excel file and used for Hypothesis testing (Appendix 1.4).

#### **IV. Results and Findings**

In order to understand the dataset at hand better, basic exploratory data analysis (EDA) was performed (Appendix 1.1). As initially mentioned the goal of this was to interpret the data statistically to numerically make sense of the data at hand. First, let's look at the distribution of the auditory features (valence, energy, danceability and acousticness) as per the genres (Figure 7). This is integral to gaining an insight into how similar the songs of certain genres are. Visually, the following inferences can be made:

- Certain genres, for example *circuit* and *guaracha* have very similar feature distribution, indicating their similarity which can be interpreted as ‘ease of transition’. This means that someone who is an active *circuit* listener wanting to get into *guaracha* will have multiple options of exploration as most of the *guaracha* songs represent similar features to circuit. Additionally, the probability that the recommendations are appropriate and liked by the user increases.
- Contrastingly, when examining genres that vary significantly in terms of their features, such as *indie triste* and *uk diy punk*, it was noted that even the songs that are the ‘most’ similar for these two genres have such high feature-wise differences that their likeability becomes highly ambiguous. For instance, the recommendations generated for an *indie triste* listener for *uk diy punk* would still be so dissimilar to his music preference that he might not enjoy them..

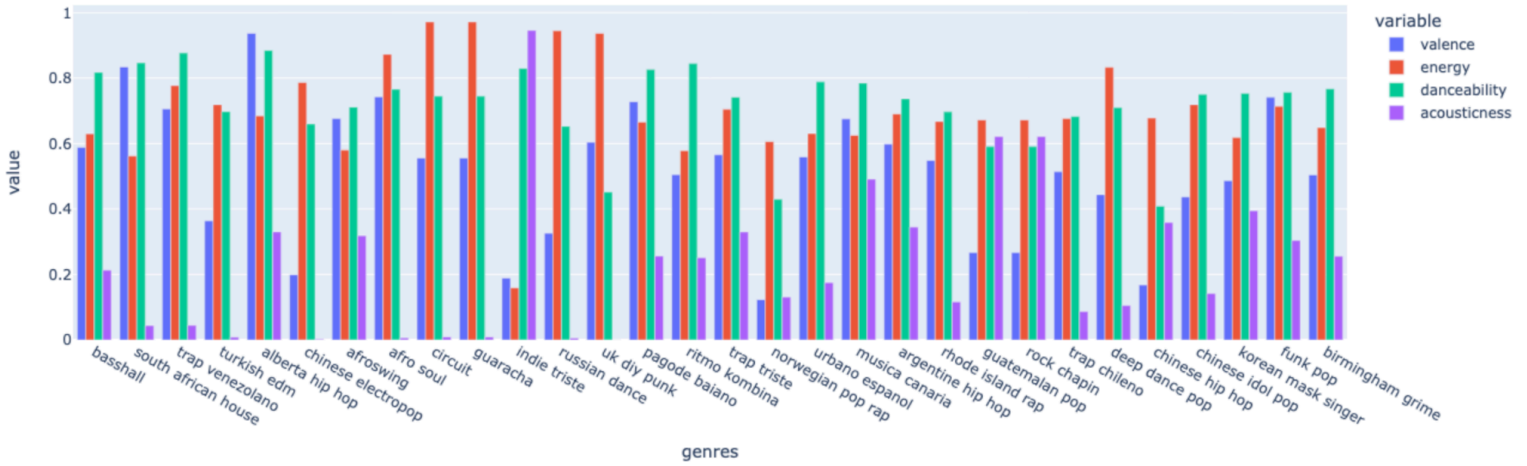


Figure 7: Genre-wise Feature Distribution (Appendix 1.1)

Although it was possible to statistically study the distribution of each feature separately, this approach might not prove to be of any use, as 'likeability', and thereby its relevance in the recommendation system, greatly depends on the combined listening experience of all the features and very little on independent features. However, it is indeed an interesting observation how the correlation of certain features translates into the production of music practically (Figure 8). For example, *energy* and *danceability* hold a strong negative correlation to *instrumentalness* as the classical songs (with high *instrumentalness*) are more calm and theatrical and less energetic and danceable.

## Correlation Matrix for Song Features

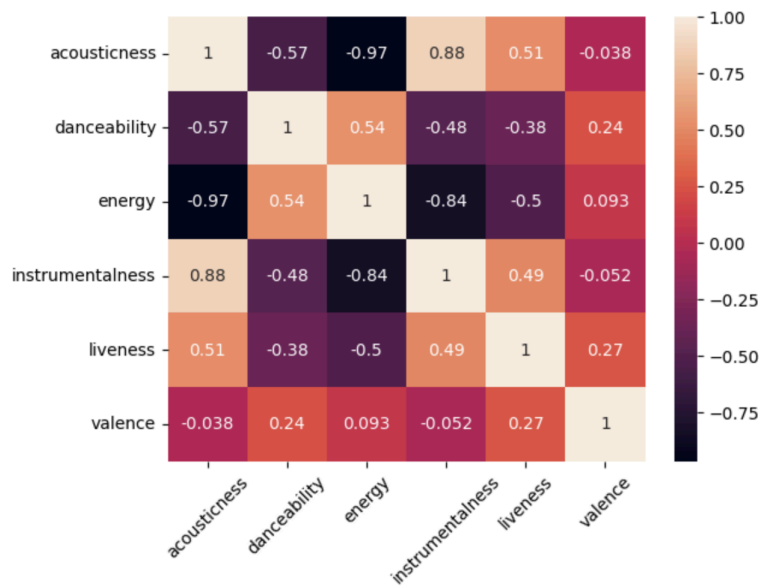


Figure 8: Correlation Matrix of features (Appendix I.1)

Let's further take a look at how the song features have changed throughout the years (Figure 11).

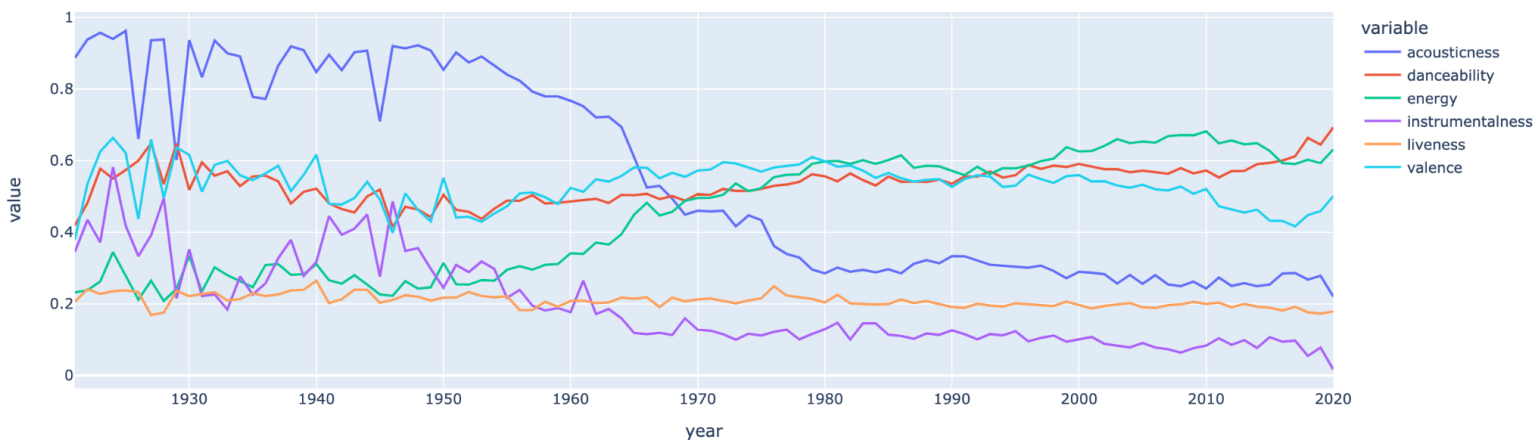


Figure 9: Song Features trend and fluctuations throughout the years (Appendix I.1)

It was observed how *instrumentalness* and *acousticness* peaked in the early 1900s and dropped in a similar pattern as the timeline moved towards the 2000s, complementing their significantly strong positive correlation of 0.88. Contrarily, *energy* remained low initially but peaked around the 1960s and continued to increase until the 2020s. Similarly, *danceability* started off higher than *energy* in the early 1900s but followed an increasing trend after the 1970s, coinciding with their moderately positive correlation of 0.55.. The strong negative correlation of -0.97 between *acousticness* and *energy* is also captured through this timeline where they both follow fairly inverse behavior throughout the years.

Next, a brief look was taken at the distribution of the main genre (after the feature engineering) (Figure 10).

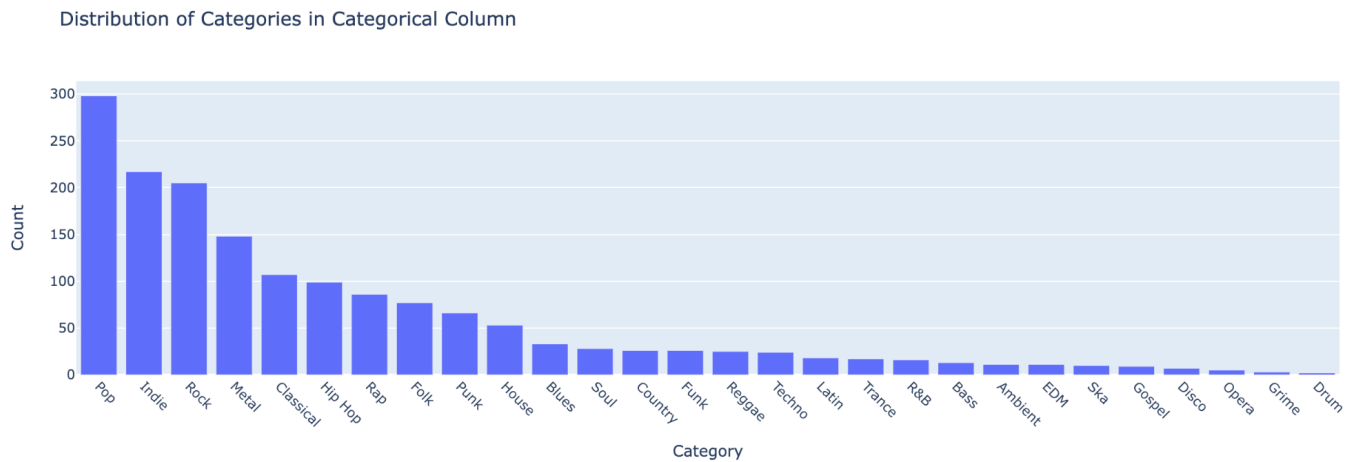


Figure 10: Main Genre Distribution (Appendix 1.1)

It was noticed that 'Pop' is the genre with the most data available. Although this might be an appropriate, but not direct, indicator of the genre's popularity, it provides a heads up that recommendations for 'Pop', followed by 'Indie', 'Rock', 'Metal', and so on, are guaranteed due to the large number of songs available to choose from while performing the recommendation algorithm. Consequently, the calculated recommendation with the highest similarity can be perceived as more accurate compared to genres like 'EDM', where there isn't a lot of data available to compare similarity against. To have a direct indication of a genre's *popularity*, the popularity feature can be conveniently used to measure this (Figure 11).

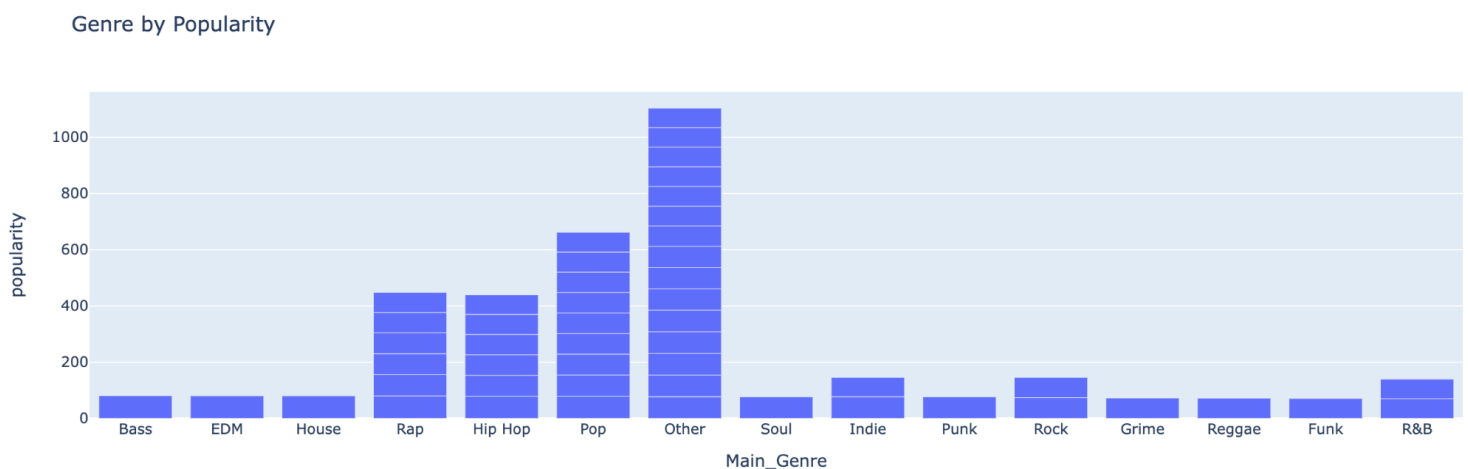


Figure 11: Popularity distribution of genres (Appendix 1.1)



It was observed that 'Pop' is the most popular genre, excluding the 'Other' category, which is a concatenation of several unclassified genres. 'Pop' is followed by 'Rap' and 'Hip Hop', while being significantly more popular than 'EDM'. Therefore, it can be concluded that the dataset's density distribution for the genres corresponds well to their respective popularity.

However, when a survey was conducted, attaining 38 responses, the results of 'Select the genre you want to get into' were as follows (Figure 12).

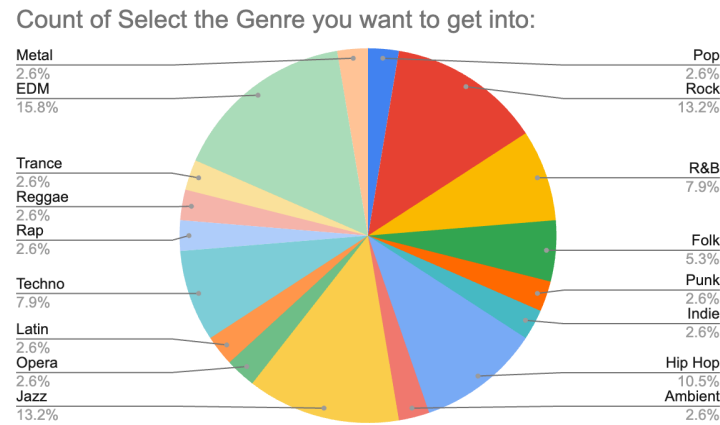
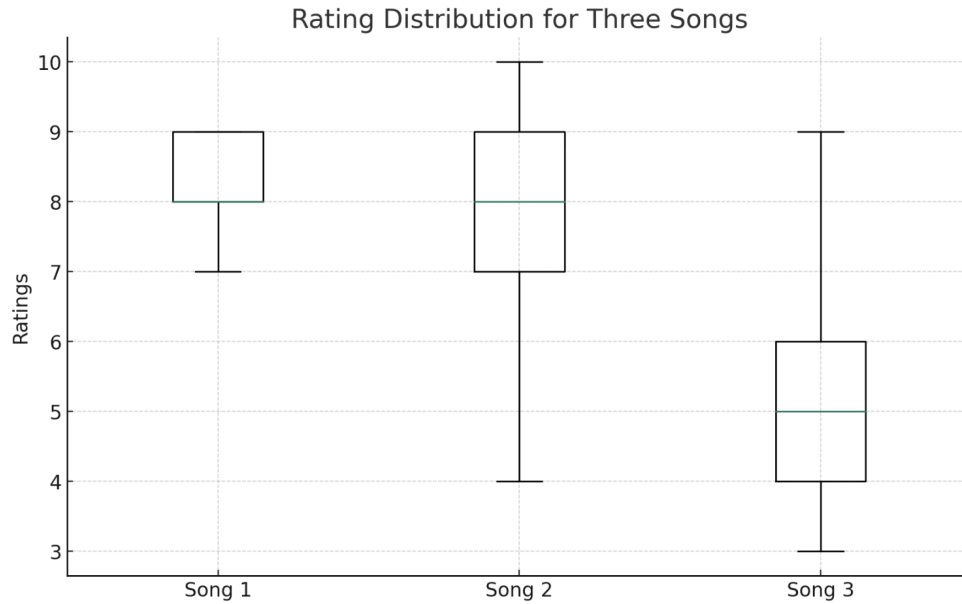


Figure 11: Pie Chart for 'Select the genre you want to get into' (Appendix 1.3)

The respondents' interest remained in discovering genres that were found to be 'unpopular' from the EDA; 15.8% and 13.2% of the respondents selected 'EDM' and 'Rock' respectively, while 'popular' genres were selected less often with merely 2.6% being interested in 'Pop'. Inconsistencies do exist however which makes this generalization not ideal; with 'Hip Hop' being popular and still selected around 10.5% of the time and some unpopular genres like 'Soul' not being selected by a single respondent (Appendix 1.3).

The main purpose of this survey was to attain user playlists and recommend to them the top 3 songs that the constructed recommendation system generates for their selected genre of interest. The respondents were then sent these recommendations and were asked to rate them on a scale of 1-10, the results of which were as follows (Figure 12).



*Figure 12: Box plot of all the ratings (Appendix 1.4)*

From the box plot, it is evident that Song 1 was rated better compared to Song 2 and Song 3. In contrast, Song 3 shows a wider spread in its ratings and includes an outlier at a higher rating, indicating more variability in how it was received by listeners. Song 2, while not performing as poorly as Song 3, still falls short of the reception and consistency of Song 1.

The average rating for all three songs per respondent was calculated and represented as a singular column to perform hypothesis testing in order to evaluate the recommendation system. A simple One Sample t-test hypothesis testing was performed with the following null and alternate hypotheses, considering 7 as a threshold for what is considered a 'high' rating:

*H0: The mean rating is equal to or less than 7. ( $\mu \leq 7$ )*

*Ha: The mean rating is greater than 7. ( $\mu > 7$ )*

This hypothesis was done using a python program and attaining the following results (Figure 13).

```
import pandas as pd
from scipy.stats import ttest_1samp

# Load the Excel file
file_path = 'Ratings.xlsx'
df = pd.read_excel(file_path)

t_statistic, p_value = ttest_1samp(df['Avg.'], 6)

# Since it's a one-sided test (testing if the mean is greater than the benchmark)
one_sided_p_value = p_value / 2

print(f'T-statistic: {t_statistic}, One-sided P-value: {one_sided_p_value}')
```

✓ 0.0s

T-statistic: 8.809016295747869, One-sided P-value: 6.460646304653986e-11

Figure 13: Code and Results from Hypothesis Testing (One-Sample t-test) (Appendix 1.5)

The t-statistic of 8.81 is substantially positive and the p-value is extremely small, far below the conventional alpha level of 0.05 or 5% (Appendix 1.5).

## V. Discussion

### i. Evaluating the Recommendation System

Let's interpret the results of the hypothesis testing. The t-statistic of 8.81 is substantially positive, suggesting that the average rating is significantly higher than the benchmark value of 7. The p-value is the probability of observing test results at least as extreme as the ones actually observed, under the assumption that the null hypothesis is true. A p-value of  $6.46 \times 10^{-11}$  suggests a very small probability (much less than 1%) of seeing the extreme of a result if the true mean were actually 7 or lower, far below the conventional alpha level of 0.05 or 5% (Appendix 1.5).

Therefore, the null hypothesis ( $H_0$ : The mean rating is equal to or less than 7. ( $\mu \leq 7$ )) is confidently rejected. The results strongly indicate that the ratings are, on average, higher than the benchmark of 7. Thereby, it can be concluded that as the ratings were fairly high on average, the recommendation system is successful in producing likable recommendations for the respondents respective genre of interest.

Further inference can be made that Song 1 and Song 2, the songs that are classified as the top recommendation, are much better than Song 3 which received significantly lower rating on average than the top 2 songs.

### ii. Limitations

There are a few limitations that emerge in the research and development of this recommendation system which are necessary to discuss in order to make the study of this research realistically applicable.

- Restrictive Sampling: The responses were collected through convenience sampling which makes the collected data highly biased culturally and geographically since the recommendation system. A more robust method such as Systematic sampling or Simple Random sampling would have promised a better representation of the population and thereby, an accurate evaluation of the recommendations. However, these methods of sampling were not used as they would require a lot of time and not promise responses with all certainty.
- Explicit Songs and Lyrics: The recommendation system merely takes into consideration the auditory similarity and not the explicit rating and lyrics of the songs in question. This might be a problem if certain words might be offensive or disliked by users. Similar concerns apply to controversial topics discussed in certain songs.
- Engineering of Main Genre: The ‘Other’ category is quite significant and was ignored (Appendix 1.2). An alternate approach of dealing with sub genres that don’t fall into a direct category of main genre is to have a similarity vector of each main genre and unclassified sub genre and use the cosine similarity index to classify them into the main genre they respectively match to the most. This method is very rigorous and requires extremely strong computational power.

These limitations indeed taint the results and findings, however, the highly positive results from the hypothesis testing performed are enough evidence that the genre-based music recommendation system constructed effectively improves user satisfaction and engagement.

### *iii. Area of Further Research*

This recommendation system serves as the basis of applying any potential kind of filter to a recommendation system. Alternative approaches towards a filter-based recommendation system include- exploring new artists, music from different cultures and languages, music from a different time-period and additionally, a more complex hybrid approach, finding the intersection between two (or more) genres, or any of the previously mentioned filters, through averaging the cosine similarity index between multiple vectors.

The recommendation system can be implemented into music platforms as an ‘Explore New Genre’ section in order to make their platform more interactive and additionally, use their rigorous database of user’s digital footprint and songs from varied genres to create results that are far more accurate. The same method can be employed through the development of a browser extension instead which uses the recommendation system to automate further recommendations restricted to a new genre of exploration.

Future enhancements would also greatly benefit from taking into account the limitations and compensating for them through the methods mentioned above in order to provide a more robust outcome.

## **VI. Conclusion**

Conclusively, this thesis not only contributes a functional enhancement to existing recommendation systems but also opens up enhanced user interaction and satisfaction in music streaming services. Through the systematic integration of users' historical preferences and a robust algorithmic framework utilizing features like danceability, tempo, and acousticness, the system has shown a notable capacity to offer personalized music recommendations that resonate well with individual tastes.

The research findings underscore the effectiveness of the recommendation system, particularly highlighted by the positive outcomes of the hypothesis testing. The t-statistic and p-value obtained from the testing strongly support the recommendation system's effectiveness, proving that users mostly appreciated the music recommended to them by the system. This validation not only confirms the system's technical efficiency but also its practical utility in enhancing user engagement and satisfaction within the music streaming industry.

The study also acknowledges several limitations, including the cultural and geographical biases inherent in the convenience sampling method used, the oversight of explicit content in songs, and the simplification of genre classification which might obscure richer, more nuanced user preferences. Future research should aim to address these gaps by adopting more rigorous sampling methods, integrating lyrical analysis to filter explicit content, and enhancing genre classification systems. Additionally, expanding the recommendation system to include filters for discovering music across different cultures, languages, and time periods can significantly enrich the scope of music discovery and personalized user experience. The pursuit of such advancements will undoubtedly continue to shape the dynamics of how music is experienced in the digital age.

## VII. Reference list

Aggarwal, C.C. and Springer International Publishing Ag (2018). Recommender Systems The Textbook. Cham Springer International Publishing Springer.

Baena-García, M., Carmona-Cejudo, J.M., Castillo, G. and Morales-Bueno, R. (2011). TF-SIDF: Term frequency, sketched inverse document frequency. [online] IEEE Xplore.  
*doi:https://doi.org/10.1109/ISDA.2011.6121796.*

Dietmar Jannach, Cambridge University Press and Al, E. (2012). Recommender systems : an introduction. Cambridge: Cambridge University Press.

Duarte, F. (2024). Music Streaming Services Stats (2023). [online] Exploding Topics. Available at:  
<https://explodingtopics.com/blog/music-streaming-stats>.

ElifTech Insights. (2021). ELIFTECH | Blog | All you need to know about a music recommendation system with a step-by-step guide to creating it. [online] Available at:  
<https://www.eliftech.com/insights/all-you-need-to-know-about-a-music-recommendation-system-with-a-step-by-step-guide-to-creating-it/>.

Giannakopoulos, T. and Pirkakis, A. (2014). Introduction to audio analysis : a MATLAB approach. Oxford: Academic Press.

Grigorescu, D.N. (2020). How to Calculate Stickiness• Metabeta. [online] Metabeta. Available at:  
<https://www.metabeta.com/blog/stickiness/#:~:text=Stickiness%20is%20the%20metric%20that> [Accessed 7 Apr. 2024].

Jure Leskovec, Anand Rajaraman and Ullman, J.D. (2020). Mining of massive datasets. Cambridge Cambridge University Press.

Malvani, V. (2022). Spotify dataset. [online] Kaggle. Available at:  
<https://www.kaggle.com/datasets/vatsalmavani/spotify-dataset>.

Ricci, F., Lior Rokach and Bracha Shapira (2015). Recommender systems handbook. New York (N.Y.): Springer, Cop.

Risdiandita, N.D. (2021). Recommendation System PART 1 — Use of Collaborative Filtering and Hybrid Collaborative — Content.... [online] Medium. Available at:

<https://towardsdatascience.com/recommendation-system-part-1-use-of-collaborative-filtering-and-hybrid-collaborative-content-in-6137ba64ad58#:~:text=The%20user%2Ditem%20interaction%20matrix>  
[Accessed 3 Feb. 2024].

Schedl, M., Knees, P., McFee, B., Bogdanov, D. and Kaminskas, M. (2015). Music Recommender Systems. Recommender Systems Handbook, pp.453–492.  
doi:[https://doi.org/10.1007/978-1-4899-7637-6\\_13](https://doi.org/10.1007/978-1-4899-7637-6_13).

Spotify (2023). Spotify — 2023 Wrapped. [online] Spotify. Available at:  
<https://newsroom.spotify.com/2023-wrapped/>.

spotipy.readthedocs.io. (n.d.). Welcome to Spotipy! — spotipy 2.0 documentation. [online] Available at:  
<https://spotipy.readthedocs.io/en/2.22.1/>.

Thorat, P., Goudar, R. and Barve, S. (2015). Survey on Collaborative Filtering, Content-based Filtering and Hybrid Recommendation System. International Journal of Computer Applications, [online] 110(4), pp.975–8887. Available at: <https://research.ijcaonline.org/volume110/number4/pxc3900760.pdf>.

Torabi, N. (2023). The Inner Workings of Spotify’s AI-Powered Music Recommendations: How Spotify Shapes Your Playlist. [online] Medium. Available at:  
<https://neemz.medium.com/the-inner-workings-of-spotifys-ai-powered-music-recommendations-how-spotify-shapes-your-playlist-a10a9148ee8d#:~:text=Spotify>.

Wright, B. (2022). The age of discovery for a music industry rebuilding after COVID: 3 in 4 people resolve to discover new genres of music in 2022. [online] [www.rpo.co.uk](http://www.rpo.co.uk). Available at:  
<https://www.rpo.co.uk/news-and-press/80-news/587-the-age-of-discovery-for-a-music-industry-rebuilding-after-covid-3-in-4-people-resolve-to-discover-new-genres-of-music-in-2022> [Accessed 3 Feb. 2024].

## VIII. Appendices

### *Appendix 1 - Files*

<https://github.com/LEon3209/SpotifyRecommendationSystem>

<https://github.com/LEon3209/HypothesisTesting>

### *Appendix 2 - Website*

<https://leon3209.github.io/Survey/>